Git & GitHub
2주차 세미나 B!BimBab

Branch & Merge — 독립된 작업 공간과 충돌(Conflict) 해결

Contents

01. Branch 란?

02. Detached HEAD

03. Merge의 두 얼굴 — Fast-forward vs 3-way

04. Conflict 해부

05. 미션 (Lv1 ~ Lv4)

1. 준비

① Git 버전 확인 2.28 이상이어야 오늘 쓰는 git init -b, git switch 가 모두 동작해요

```
$ git --version
git version 2.xx.x   ← 2.28 이상인지 확인
```

② 설정 두 줄

```
$ git config --global merge.conflictStyle diff3
$ git config --global alias.lg "log --oneline --graph --all"
```

diff3 : 충돌이 나면 공통 조상(원래 모습)까지 함께 보여줌
alias.lg : git lg 한 줄로 커밋 그래프 보기

1주차 복습 퀴즈

Q. 커밋은 무엇을 저장할까요?

① 바뀐 부분만 (변경분)

② 그 시점 프로젝트 전체의 모습 (스냅샷)

손 들어서 답해보기!

정답: 그 시점의 스냅샷

커밋 = 변경분이 아니라 그 시점 프로젝트 전체의 스냅샷

그리고 하나 더! 커밋은 자기 부모 커밋을 기억한다



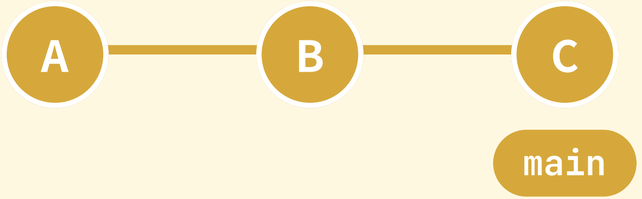
이 "부모를 기억한다"가 오늘 배울 **그래프 · merge base · 3-way merge** 설명 전부의 전제!

01

Branch 란?

01 왜 브랜치가 필요할까?

지금까지 열심히 작업해서 C 커밋까지 왔다고 해보자.



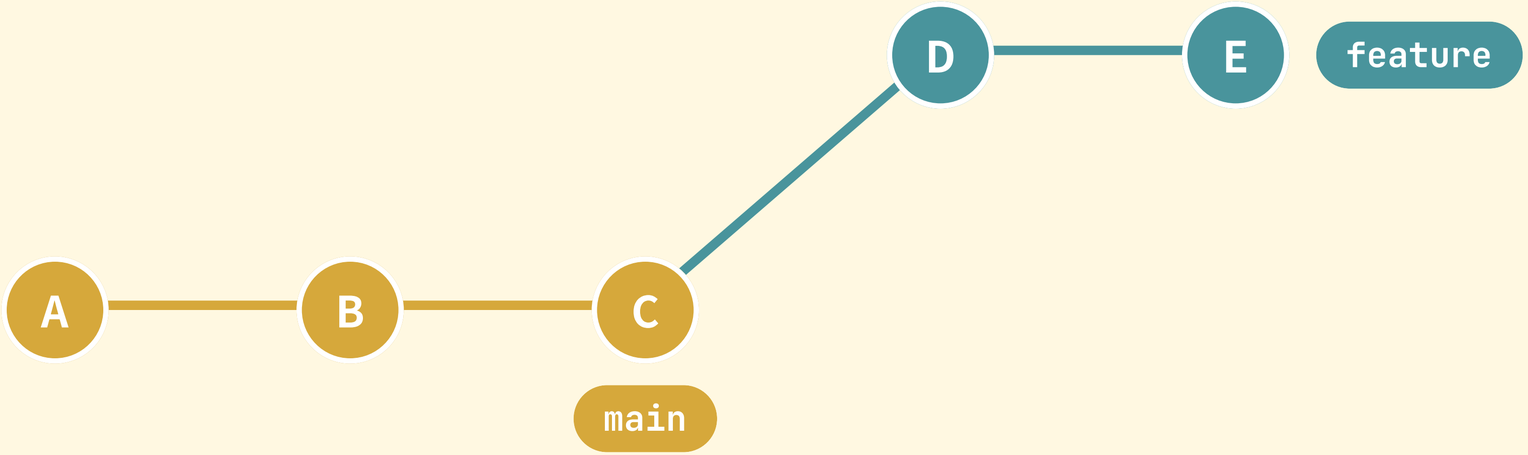
"로그인 기능을 추가해보고 싶은데...
혹시 망하면 어떡하지?"

기존 작업은 그대로 둔 채, 다른 방향으로 작업할 수 있는 길을 하나 만들자

→ 이게 바로 브랜치(Branch)

01 Branch 란?

브랜치 = 하나의 프로젝트에서 작업의 흐름을 나누는 방법



main : 원래 작업 흐름
feature : 새로 갈라진 작업 흐름

feature에서 무슨 짓을 해도
main은 안전!

기존 작업을 건드리지 않고, 새로운 기능을 마음 놓고 실험할 수 있다

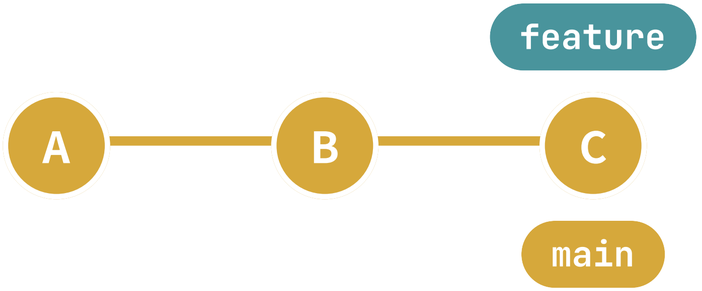
01 그럼 프로젝트 복사랑 똑같은 거 아냐?

프로젝트 통째로 복사

```
project/
project-copy/
project-copy2/
project-진짜최종/
```

파일 전체를 복사 → 용량도 커지고,
어떤 폴더가 최신인지 헷갈림

Git 브랜치



"여기서부터 다른 방향으로 가겠다"는
이름표(표시) 하나만 붙이는 것

그래서 브랜치는 부담 없이 만들고, 부담 없이 지울 수 있다!

01 브랜치 명령어

```
$ git branch
```

현재 브랜치 목록 보기

* 표시가 붙은 게 지금 내가 있는 브랜치

```
$ git branch feature
```

feature 라는 작업 흐름을 하나 만듦

만들기만 할 뿐, 아직 feature로 이동한 건 아님!

```
$ git switch feature
```

feature 브랜치로 이동

이제부터 커밋하면 feature 쪽에 쌓인다

```
$ git switch -c feature
```

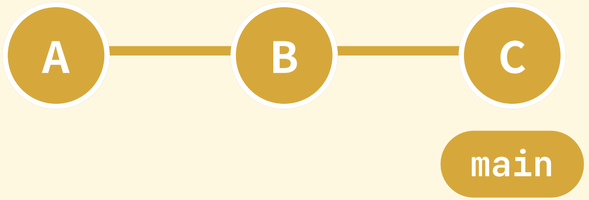
만들기 + 이동을 한 번에

= git branch feature + git switch feature

참고) 예전 방식인 git checkout feature / git checkout -b feature 도 같은 역할

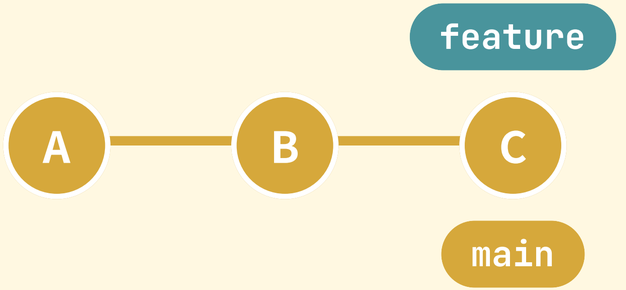
01 직접 해보기 ① 브랜치 만들고 이동하기

처음




```
$ git switch -c feature
```

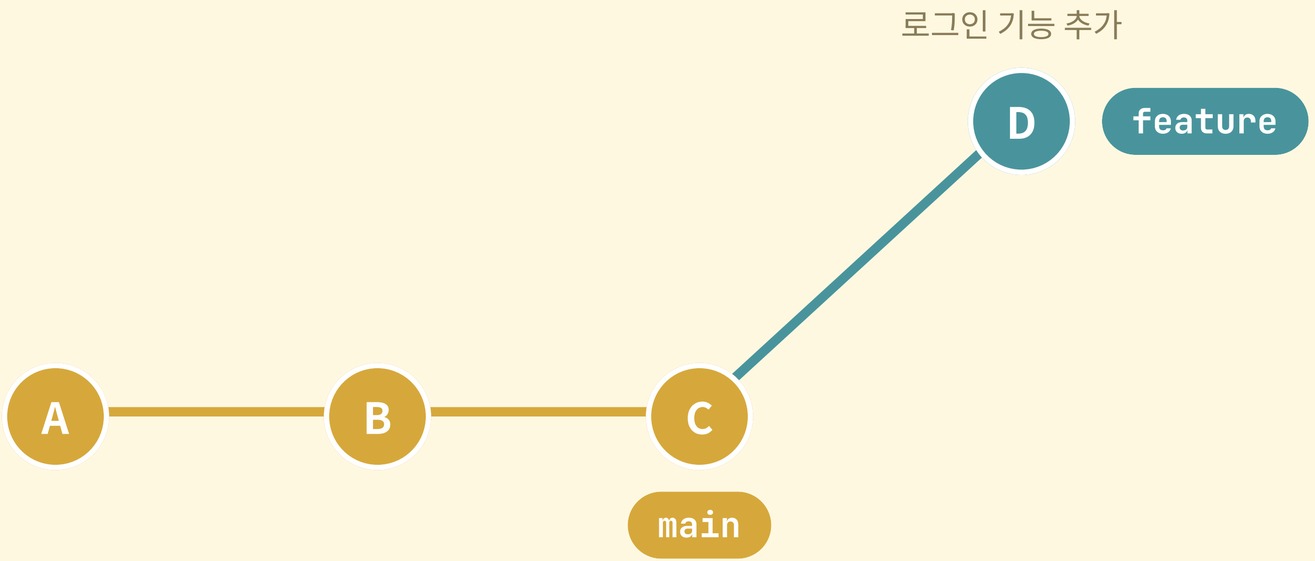
git switch -c feature 후



아직 새 커밋이 없으니 main과 feature 둘 다 C를 가리킨다
(지금 내가 있는 곳은 feature)

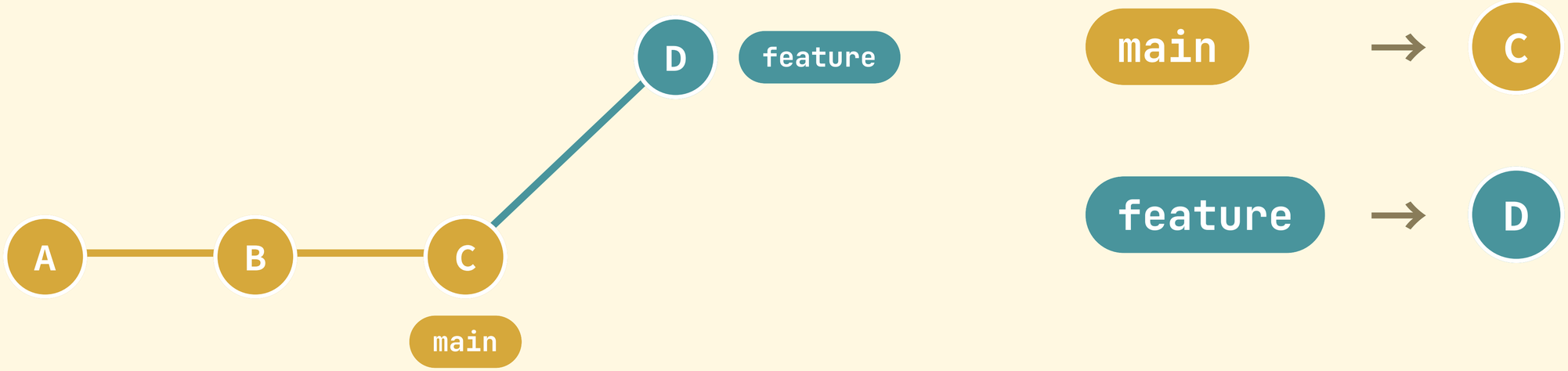
01 직접 해보기 ② feature에서 커밋하기

```
# 로그인 기능 코드 작성 후
$ git add .
$ git commit -m "로그인 기능 추가"
```



main은 그대로 C,
feature만 D로
한 칸 앞으로!

01 그런데 Git은 이걸 어떻게 관리할까?



Git에서 브랜치는 프로젝트 파일을 복사해 둔 게 아니라,

특정 커밋을 가리키는 '이름(포인터)'이다.

01 .git/refs/heads 들여다보기

브랜치는 여기에 저장된다

```
.git/
└── refs/
    └── heads/
        ├── main
        └── feature
```

cat : 파일 내용을 출력하는 명령어

```
$ cat .git/refs/heads/main
203fb3589b5ed4224e3de25c82110414c675d035

$ cat .git/refs/heads/feature
9c1e7a42d0b86e3a5c2d4b7e1f09a8c6d3b2e150
```

(해시 값은 예시)

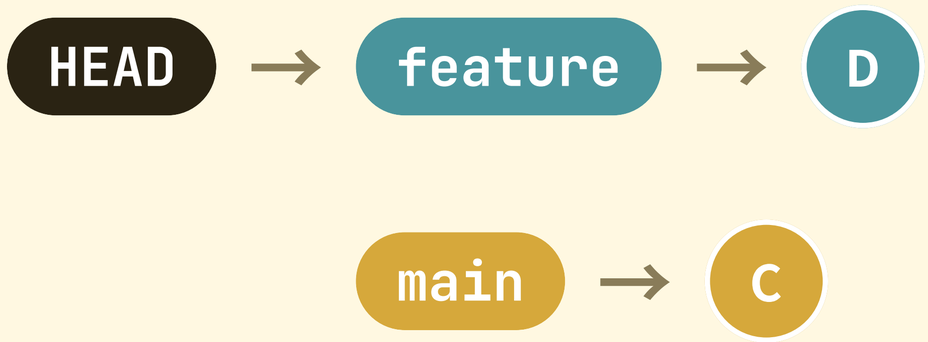
브랜치 파일 안에는 커밋 해시(40자리) 딱 한 줄만 들어있다

→ 브랜치 만들기 = 이 작은 파일 하나 만들기 → 그래서 가볍고 빠르다

→ 커밋하면 이 파일 속 해시만 새 커밋으로 바뀐다 = 브랜치가 앞으로 움직이는 원리

01 HEAD — 지금 나는 어느 브랜치에 있을까?

그럼 지금 내가 어느 브랜치에서 작업하고 있는지는 어떻게 알까?




```
$ cat .git/HEAD
ref: refs/heads/feature
```

HEAD = 지금 내가 서 있는 브랜치를 가리키는 포인터

→ git switch main 하면 HEAD가 main을 가리키도록 바뀐다

→ 커밋하면 HEAD가 가리키는 브랜치가 새 커밋으로 이동한다

01 git lg 로 한눈에 확인하기

```
$ git lg
* D (HEAD -> feature)
* C (main)
* B
* A
```

HEAD -> feature
지금 feature 브랜치에서 작업 중,
feature는 D를 가리킴

(main)
main 브랜치는 여전히 C를 가리킴

git lg : 커밋 그래프를 한눈에 보기 좋게 보여주는 log 별칭(alias)

지금까지 그린 그림과 똑같다!
HEAD → feature → D / main → C

01 시연 1 — 브랜치는 파일이다

```
$ mkdir branch-demo && cd branch-demo
$ git init
$ echo "hello" > a.txt
$ git add . && git commit -m "first"

$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
3f2a9c1...   (40자 해시)
```

주의

커밋이 하나도 없으면 refs/heads/main 파일이 아직 없어요.
반드시 커밋부터!

main 브랜치 의 정체 = 해시 40자가 적힌 텍스트 파일 하나

HEAD = 지금 내가 어느 브랜치에 있는지 적어둔 파일

01 시연 2 — 브랜치 만들기 = 파일 하나 생성

```
$ git branch feature
$ cat .git/refs/heads/feature
3f2a9c1...   ← main과 똑같은 해시
$ ls .git/refs/heads
feature  main
```

코드가 복사되는 게 아니라 41바이트짜리 파일 하나가 생길 뿐

→ 그래서 Git에서는 브랜치를 부담 없이 만들고 버린다

→ 1주차에 본 기존 버전관리 시스템(폴더째 복사)의 한계와 다른 점

01 시연 3 — 이동하면 HEAD 파일만 바뀐다

```
$ git switch feature
$ cat .git/HEAD
ref: refs/heads/feature

$ echo "feature work" >> a.txt
$ git commit -am "feature commit"
$ cat .git/refs/heads/feature  # 해시가 바뀜
$ cat .git/refs/heads/main  # 그대로
$ git lg
```

switch
→ .git/HEAD 파일 내용만 바뀜

commit
→ HEAD가 가리키는 브랜치 파일의
해시만 바뀜

커밋하면 HEAD가 가리키는 브랜치 파일의 해시만 새 커밋으로 갱신된다
이 한 문장을 이해하면 이후 모든 게 쉬워져요

01 switch vs checkout

git checkout 은 브랜치 이동, 파일 복원 등 역할이 너무 많았어요
→ Git 2.23부터 **switch** (이동) 와 **restore** (복원) 로 나뉨
인터넷 자료엔 checkout이 많이 보이지만, 우리는 switch를 씁니다

자주 쓰는 형태

```
$ git branch              # 목록
$ git switch 이름         # 이동
$ git switch -c 이름      # 만들면서 이동
$ git branch -d 이름      # 병합된 브랜치 삭제
$ git branch -D 이름      # 강제 삭제
```

01 자주 막히는 곳 — 커밋 안 하고 switch

```
$ echo "작업 중..." >> a.txt  # 커밋 안 함
$ git switch main
error: Your local changes ... would be overwritten
# 또는 에러 없이 변경사항이 main으로 따라옴
```

워킹 디렉토리

작업 중인 파일은 여기 소속!

스테이징 영역

git add 한 것

저장소 (.git)

커밋 = 브랜치가 가리키는 곳

작업 중인 파일은 브랜치 소속이 아니라 워킹 디렉토리 소속

→ switch 전에 git status 로 확인하고, 커밋한 뒤 이동하기

02 Detached HEAD

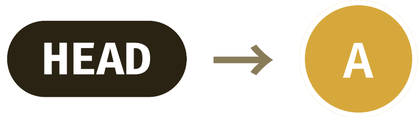
02 Detached HEAD 란?

평소 — 브랜치에 붙어 있음




```
$ cat .git/HEAD
ref: refs/heads/main
```

Detached HEAD — 떨어져 나옴




```
$ cat .git/HEAD
3f2a9c1...   ← ref: 없이 해시가 직접!
```

HEAD 파일에 브랜치 이름 대신 해시가 직접 들어간 상태

= 브랜치라는 **이름표 없이 커밋만 붙잡고 있는** 상태

02 시연 — 이름표 없이 커밋하기

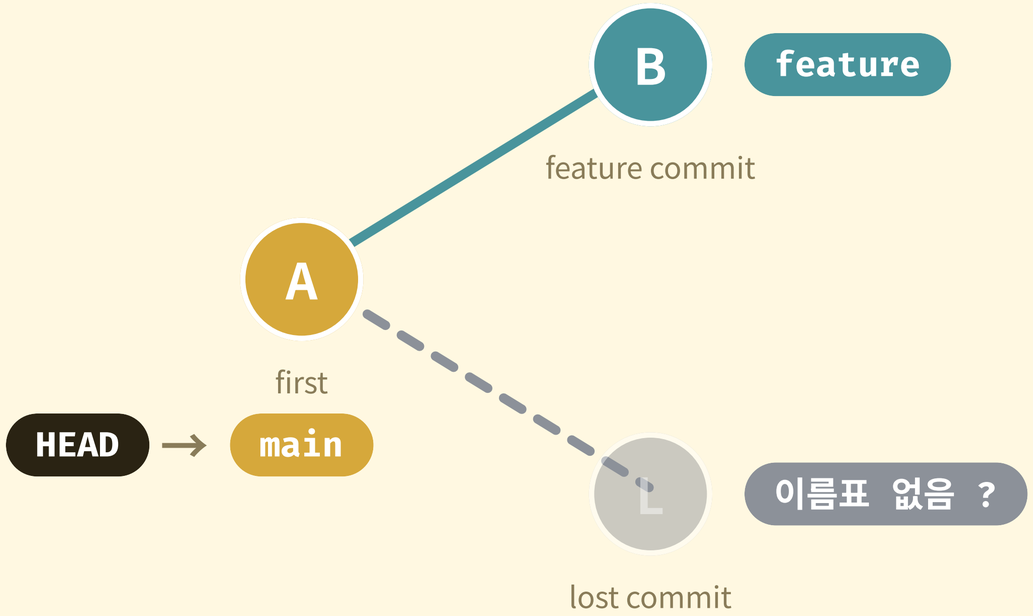
```
$ git lg  # first 커밋 해시 확인
$ git switch --detach <first해시>
$ cat .git/HEAD
3f2a9c1...   ← ref: 가 아니라 해시

$ echo "lost" > b.txt
$ git add . && git commit -m "lost commit"
$ git switch main
$ git lg  # lost commit이 안 보인다?!
```

git switch --detach <해시>
브랜치 없이 특정 커밋으로 이동
→ HEAD가 커밋을 직접 가리킴

HEAD 파일에 브랜치 이름 대신 해시가 직접 들어가 있다
→ 이 상태에서 만든 커밋은 main으로 돌아오면 그래프에서 안 보인다

02 커밋이 사라졌다…?




```
$ git switch main
$ git lg
* b2c4d1e (feature) feature commit
* 3f2a9c1 (HEAD -> main) first
# lost commit 은 어디에도 없다
```

main으로 돌아오면 이름표 없는 커밋은 그래프에서 사라져요

하지만 이 커밋은 사실 아직 살아있고, git switch -c 브랜치이름 만들어주면 됩니다! 이미 놓쳤다? 그럴 때의 명령어 각자 찾아보기!

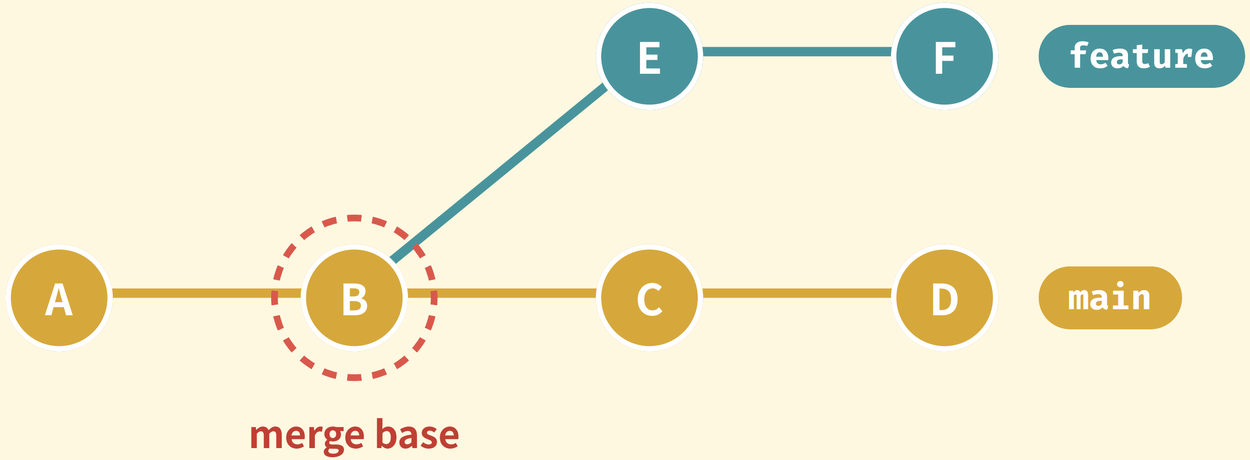
03

Merge 의 두 얼굴

Fast-Forward vs. 3-way

03 merge base — 공통 조상

merge base = 두 브랜치가 갈라지기 직전의 공통 조상 커밋



명령으로 직접 확인

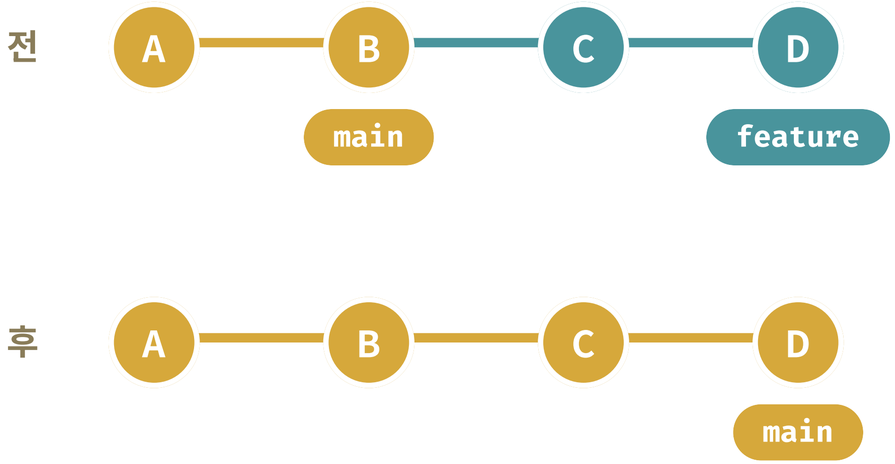
```
$ git merge-base main feature
B의 해시가 나옴
```

Merge는 항상 "갈라진 지점(base)에서부터 각자 무엇이 바뀌었나"를 본다

03 Merge의 두 얼굴

Fast-forward

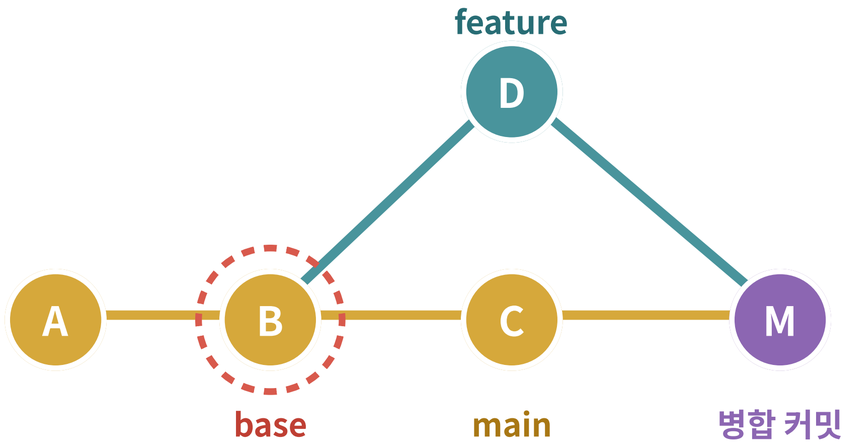
main이 갈라진 뒤 아무것도 안 했다면



main 포인터를 앞으로 옮기기만 하면 끝

3-way merge

양쪽 다 커밋이 쌓였다면



base · main · feature 세 스냅샷을 비교해서
새 병합 커밋(M)을 만든다

"3-way" = base까지 세 개를 비교한다는 뜻!

03 시연 ① Fast-forward

```
$ git switch -c ff-test
$ echo "ff" > c.txt && git add . && git commit -m "ff work"
$ git switch main
$ git merge ff-test
Updating 3f2a9c1..8d1e4b2
Fast-forward
```

출력에 **Fast-forward** 가 보이면 → 새 커밋 없이 main 포인터만 앞으로 옮긴 것

03 시연 ② 3-way merge

```
$ git switch -c three-way
$ echo "tw" > d.txt && git add . && git commit -m "three-way work"
$ git switch main
$ echo "main" > e.txt && git add . && git commit -m "main work"
$ git merge three-way
# 에디터가 열림 → 저장하고 닫기
Merge made by the 'ort' strategy.
$ git lg
```

에디터가 열리면?

VS Code가 병합 커밋 메시지를 물어봐요

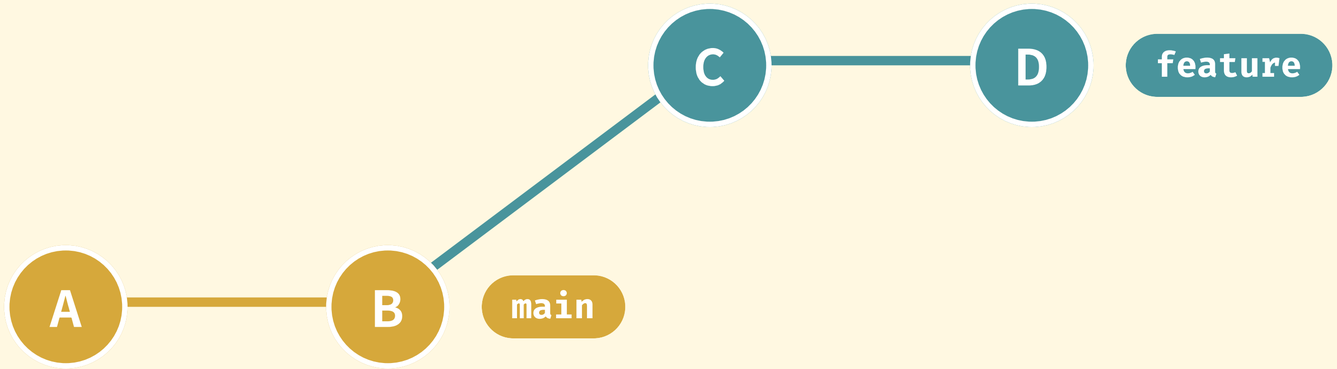
→ 기본 메시지 그대로 저장하고 탭 닫기

에디터 없이 하려면
git merge three-way --no-edit

Merge made by the 'ort' strategy. → 3-way로 새 병합 커밋이 만들어졌다

03 예측 퀴즈 1 / 3

main에서 git merge feature 하면?



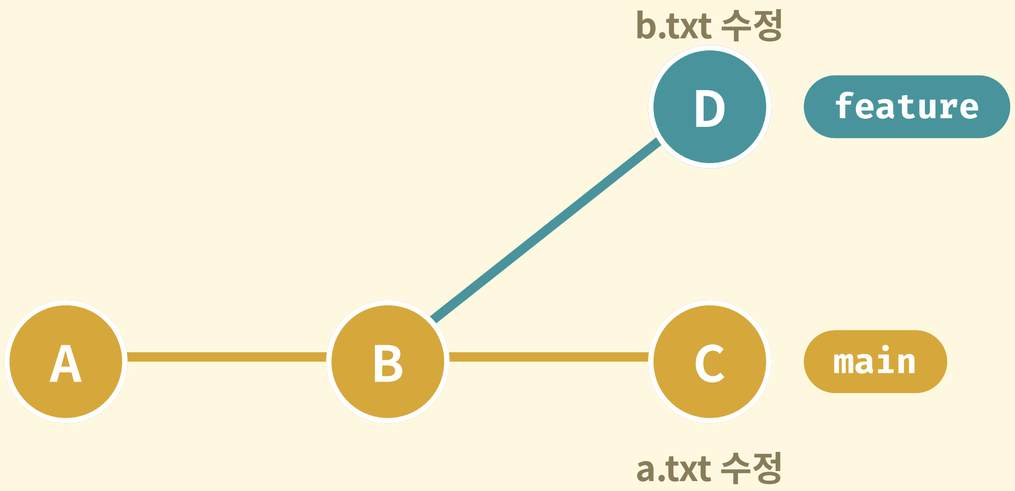
① Fast-forward

② 3-way (충돌 없음)

③ 3-way + 충돌

03 예측 퀴즈 2 / 3

main에서 git merge feature 하면?



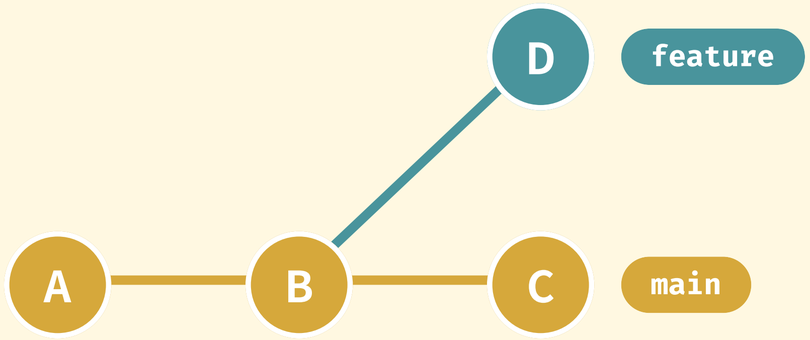
① Fast-forward

② 3-way (충돌 없음)

③ 3-way + 충돌

03 예측 퀴즈 3 / 3

main에서 git merge feature 하면?




```
# main 쪽 recipe.txt
1. 밥
2. 시금치 나물
3. 고추장
4. 참기름
```

```
# feature 쪽 recipe.txt
1. 밥
2. 나물
3. 고추장 두 스푼
4. 참기름
```

원래(base): 1. 밥 / 2. 나물 / 3. 고추장 / 4. 참기름 → 바로 붙어 있는 2번, 3번 줄을 각각 수정

① **Fast-forward**

② **3-way (충돌 없음)**

③ **3-way + 충돌**

03 예측 퀴즈 정답

Q1 Fast-forward

main은 그대로, feature에만 커밋 2개

Q2 3-way, 충돌 없음

양쪽에 커밋, 서로 다른 파일 수정

Q3 3-way + 충돌 발생!

같은 파일의 바로 붙어 있는 두 줄을 각각 수정

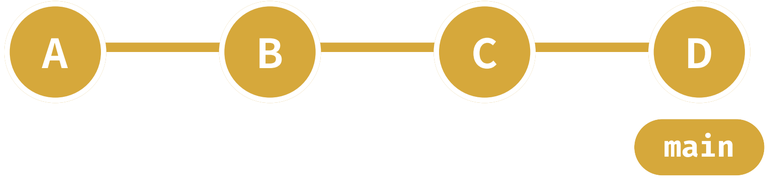
"다른 줄이니까 괜찮다"? → NO!

Git은 **인접한 줄의 변경도 충돌**로 처리한다

03 --no-ff — 병합 기록 남기기

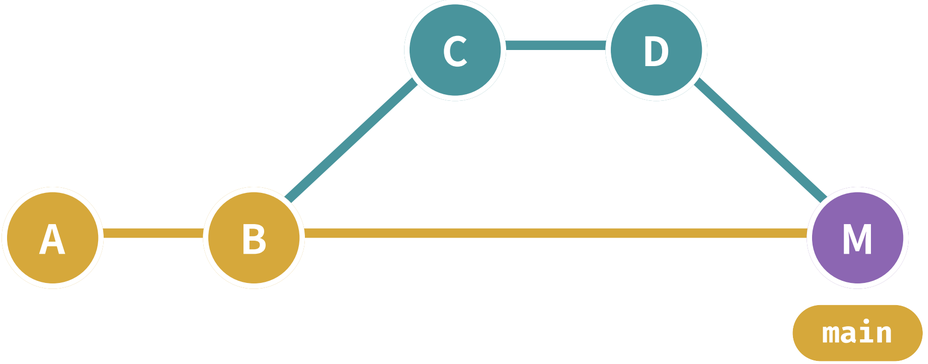
```
$ git merge --no-ff feature
```

git merge feature (FF)



일자 그래프 → 언제 합쳐졌는지 안 보임

git merge --no-ff feature



FF가 가능해도 병합 커밋 M을 만듦

"이 기능이 언제 합쳐졌는지"를 이력에 남기는 옵션

→ 6주차 팀 규칙에서 다시 나와요!

04 Conflict

충돌 해부

04 라이브 충돌 만들기

```
$ git switch -c conflict-a
$ echo "A의 문장" > f.txt && git add . && git commit -m "A"
$ git switch main
$ echo "main의 문장" > f.txt && git add . && git commit -m "main"
$ git merge conflict-a
CONFLICT (add/add): Merge conflict in f.txt
Automatic merge failed; fix conflicts and then commit the result.
$ git status
```

같은 파일 f.txt를 양쪽에서 서로 다르게 만들었다

→ 어느 쪽이 맞는지 Git이 대신 결정할 수 없으니 **사람에게 물어본다** = 충돌(Conflict)

04 충돌 마커 읽는 법

diff3 설정 덕분에 f.txt가 이렇게 보여요

```
<<<<<<< HEAD
main의 문장
||||||| 공통 조상
=======
A의 문장
>>>>>>> conflict-a
```

위 = 지금 내 브랜치 (HEAD)

가운데 = 원래 모습 (base, 공통 조상)

아래 = 합치려는 브랜치 (conflict-a)

충돌 해결이란? base와 비교해서 각자 무엇을 바꾸려 했는지 보고,
둘의 의도를 모두 살리는 코드를 사람이 직접 쓰는 것

가장 흔한 실수: 마커 줄(<<<<<<<, |||||||, =======, >>>>>>>)을 지우는 걸 잊는 것!

04 충돌 해결 4단계

1
```
$ git status
```
무엇이 충돌났는지 확인

2
```
(파일 수정)
```
마커 제거, 최종 내용 직접 작성

3
```
$ git add f.txt
```
"해결했다"는 표시

4
```
$ git commit --no-edit
```
병합 완료

탈출법 충돌 도중 겁나면 → **git merge --abort**
merge 하기 전 상태로 완전히 돌아갈 수 있어요

04 충돌 3종 — git status로 구분하기

```
both modified:   menu.txt      ← 같은 줄 수정
deleted by them: notes.txt     ← 한쪽 삭제, 한쪽 수정
both added:      extra.txt      ← 양쪽이 같은 이름으로 생성
```

modify/delete (notes.txt)는 마커가 없어서 다들 당황해요

해결법: 살릴 거면 add, 지울 거면 rm

```
$ git add notes.txt   # 살리기
$ git rm notes.txt    # 삭제 확정
```

menu.txt, extra.txt 는 마커를 보고 직접 정리 → git add

04 VS Code로 충돌 해결하기

VS Code에서 충돌 파일을 열면 이런 버튼이 떠요

Resolve in Merge Editor

→ 누르면 양쪽 변경을 나란히 보고, 체크박스로 골라 합칠 수 있음

도구는 편하지만, 마커를 직접 읽을 줄 알아야
도구가 틀릴 때 대응할 수 있다!

05 미션

05 미션 안내

Lv1 그래프 복제

Lv2 병합 방식 증명

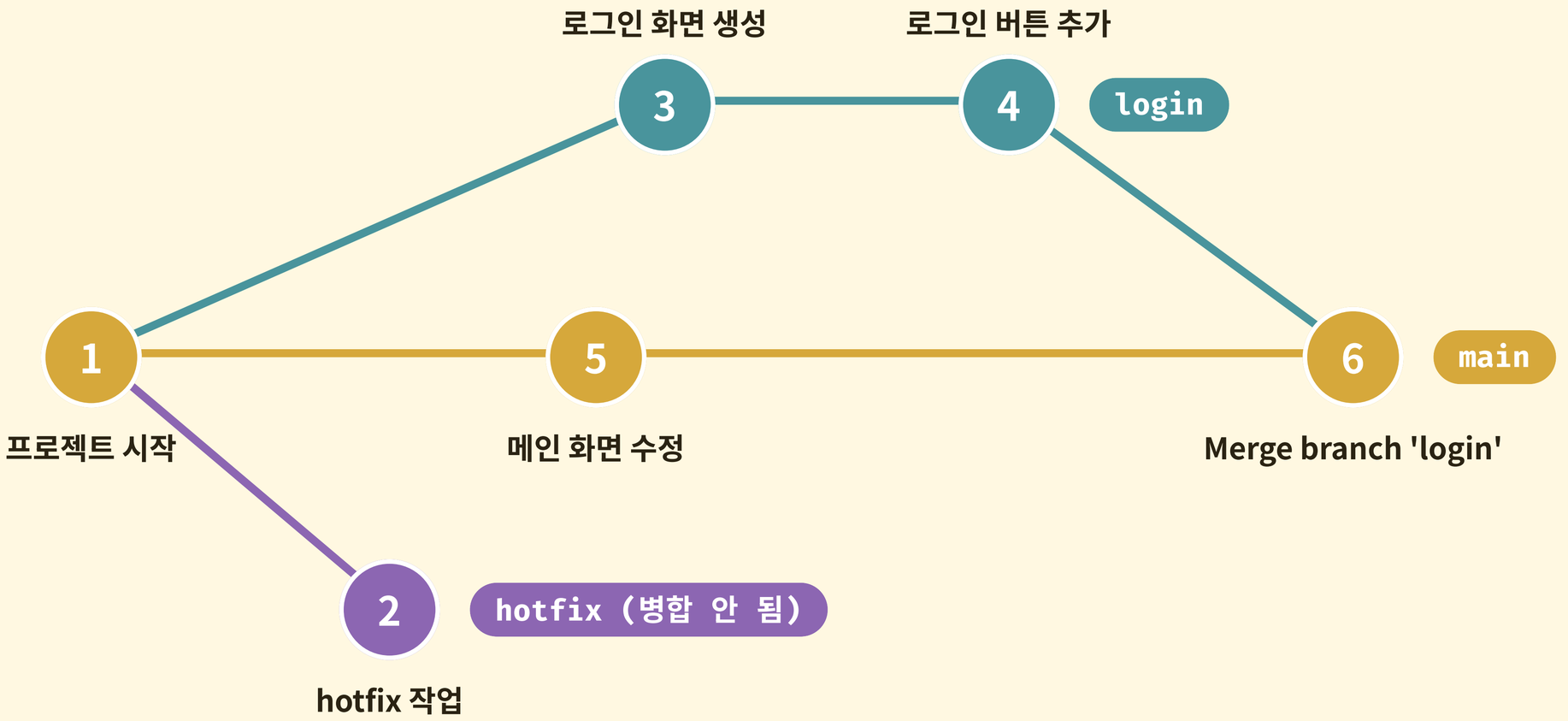
Lv3 충돌 3종 해결

Lv4 보스: 사라진 커밋 구출

미션 슬라이드에는 **목표만** 있어요. 명령어는 직접 찾아보기!
막히면 손 들기 → 힌트를 하나씩 풀어드려요
Lv3 스크립트는 처음부터 다시 만들어지니 망하면 **bash setup.sh** 다시 실행하면 끝

Lv1 그래프 복제

git lg 결과가 이 그래프와 똑같이 나오도록 만들어 보세요



숫자 = 커밋 순서

Lv2 병합 방식 증명

① Fast-forward 병합 하나 만들기
② 3-way 병합 하나 만들기
③ git lg 결과 캡처
④ "왜 이쪽은 선이 갈라지지 않았는지" 한 문장으로 설명

채점 포인트는 명령어가 아니라 **설명**!

Lv3 충돌 3종 해결

```
$ cd conflict-lab
$ git merge spicy
```

해결 기준

`menu.txt`	**"고추장 두 스푼" 과 "간장" 둘 다 들어가게**
`notes.txt`	**살리기**
`extra.txt`	**두 문장 모두 남기기**

확인: cat으로 세 파일 내용 보기 → 마커 기호가 하나라도 남아 있으면 실패!

망하면 bash setup.sh 다시 실행해서 처음부터

Lv4 보스 — 사라진 커밋 구출 작전

BOSS

detached HEAD에서 커밋을 하나 만들고,
main으로 돌아온 뒤,
그 커밋을 rescue 브랜치로 되살려라.

힌트는 막힌 사람에게만 딱 하나!

[정답] Lv1 그래프 복제

```
$ git init graph-mission && cd graph-mission
$ echo 1 > a.txt && git add . && git commit -m "프로젝트 시작"
$ git switch -c hotfix && echo h > h.txt && git add . && git commit -m "hotfix 작업"
$ git switch main
$ git switch -c login
$ echo l > l.txt && git add . && git commit -m "로그인 화면 생성"
$ echo b >> l.txt && git commit -am "로그인 버튼 추가"
$ git switch main
$ echo 2 >> a.txt && git commit -am "메인 화면 수정"
$ git merge login --no-edit
```

[정답] Lv2 · Lv3

Lv2 — 03 시연(FF / 3-way)과 동일. 설명 예: "FF는 main 포인터만 앞으로 옮겨서 새 커밋이 없으니 선이 갈라지지 않는다"

```
$ cd conflict-lab
$ git merge spicy
# menu.txt 마커 제거 후 "2. 고추장 두 스푼" / "3. 간장" 등으로 정리
# extra.txt 마커 제거 후 "순한맛 버전" / "매운맛 버전" 두 줄
$ git add menu.txt extra.txt notes.txt
$ git commit --no-edit
$ git lg
```

[정답] Lv4 보스

```
$ git branch rescue <경고에 나온 해시>
$ git lg
```

힌트는 딱 하나: "Git이 말해준 걸 다시 읽어보세요"

main으로 switch할 때 뜬 경고 메시지에 해시와 살리는 방법이 그대로 나와요

터미널을 clear해서 메시지를 날린 사람은 못 풀어요 → 그게 바로 4주차 reflog!